Meta-Learning

From Few-Shot Classification to Learning to Learn

Comprehensive Lecture Notes

Lectures 1–6 (January 5 – January 22)

IIIT Delhi

Compiled: February 26, 2026

Contents

1 Introduction and Motivation (January 5) 3
- 1.1 What is Meta-Learning? 3
- 1.2 Challenges in ML/DL/LLMs 3
  - 1.2.1 Challenge 1: Learning with Less Data (Few-Shot Learning) 3
  - 1.2.2 Challenge 2: Continual Learning 3
  - 1.2.3 Challenge 3: Distribution Shift 3
  - 1.2.4 Challenge 4: Online Learning 4
- 1.3 Generalization and Intelligence 4
- 1.4 AI and Skill Acquisition 4

2 From Classification to Attention-Based Few-Shot Learning (January 8) 5
- 2.1 Motivating Example: Customer Purchase Prediction 5
- 2.2 Step 1: Direct Estimation with Indicator Functions 5
- 2.3 Step 2: Relaxation to Neighbors (ε-Ball / k-NN) 5
- 2.4 Step 3: Distance-Weighted Prediction 6
- 2.5 Step 4: Softmax Formulation 6
- 2.6 Step 5: The Attention Connection 6
- 2.7 Step 6: Generalizing with Separate Encoders 7
- 2.8 Step 7: The Master Equation 7
- 2.9 Training via Representation Learning: Contrastive Loss 7

3 Generalization Theory and Representation Learning (January 12) 8
- 3.1 Recap: Soft k-NN and Training Approaches 8
  - 3.1.1 Approach 1: Direct (End-to-End) Training 8
  - 3.1.2 Approach 2: Contrastive Training 8
- 3.2 Task Structure and Cross-Task Generalization 8
- 3.3 The No Free Lunch Theorem 8
- 3.4 Architectures for the Embedding Function f_θ 9
  - 3.4.1 Unsupervised / Self-Supervised (Using Only X) 9
  - 3.4.2 Sequence Models (for Sequential Inputs) 9
- 3.5 Task Representation 9

4 Representation Learning Methods (January 15) 10
- 4.1 Problems with Standard Contrastive Loss 10
- 4.2 Contrastive Loss Using Noise 10
- 4.3 Reconstructive Representation Learning 10
  - 4.3.1 Variant 1: Compressive Autoencoder (Vanilla AE) 11
  - 4.3.2 Variant 2: Denoising Autoencoder 11
  - 4.3.3 Variant 3: Diffusion Models 11
  - 4.3.4 Variant 4: Variational Autoencoder (VAE) 11
- 4.4 VAE Loss Function 12
- 4.5 Sequence Architectures for f_θ 12

5 Sequence Models, Attention, and the Meta-Learning Framework (January 19) 13
- 5.1 MLP Review and Universal Approximation 13
- 5.2 Recurrent Neural Networks (RNN) 13
  - 5.2.1 The Vanishing/Exploding Gradient Problem 13
- 5.3 Long Short-Term Memory (LSTM) 14

5.4 Gated Recurrent Unit (GRU) . . . 14
5.5 Attention Mechanism . . . 14
5.6 Machine Translation: Encoder-Decoder with Cross-Attention . . . 15
5.7 Transformers . . . 15
5.8 The Formal Meta-Learning Framework . . . 15
5.9 The Meta-Learning Philosophy . . . 16

6 Matching Networks and MAML (January 22) 17
6.1 Matching Networks: Full Formalization . . . 17
6.2 Training Loss: Cross-Entropy . . . 17
6.3 Episodic Training Loop . . . 17
6.4 Context-Aware Matching Networks . . . 17
6.5 General Meta-Learning Loop . . . 18
6.6 MAML: Model-Agnostic Meta-Learning . . . 18
6.6.1 The MAML Algorithm . . . 19
6.6.2 Understanding the Two Loops . . . 19
6.6.3 Geometric Interpretation . . . 20
6.7 Matching Networks vs. MAML: Comprehensive Comparison . . . 20

Summary: The Complete Arc 21

Lecture 1: Introduction and Motivation (January 5)

What is Meta-Learning?

Meta-learning, also known as **learning to learn**, is a subfield of machine learning where the goal is to train a system that can learn new tasks efficiently from a small number of examples. Unlike conventional machine learning, where a model is trained from scratch on a large dataset for a single task, meta-learning aims to leverage experience across many tasks so that the system can rapidly adapt to new, previously unseen tasks.

Core Idea

Goal: Learn with less data.
Mechanism: Train a system to learn with less data.
Key Connection: This is intimately related to *few-shot learning* — the ability to classify or predict from only a handful of labeled examples.

There is a deep connection between meta-learning and **in-context learning (ICL)** in large language models. When an LLM receives a few examples in its prompt and then classifies a new input, it is performing a form of few-shot meta-learning at inference time — the model has "learned to learn" from its massive pretraining.

Challenges in ML/DL/LLMs

The lecture identifies several core challenges that motivate the meta-learning framework:

Challenge 1: Learning with Less Data (Few-Shot Learning)

Standard deep learning requires thousands to millions of labeled examples. In many real-world settings — rare diseases, new product categories, endangered species — such data simply does not exist. Meta-learning directly addresses this by learning representations and strategies that transfer across tasks.

Challenge 2: Continual Learning

When a model has been trained on some data and new data arrives, we face a fundamental tension:

- **Fine-tuning** on new data risks *catastrophic forgetting* — the model loses its ability to perform well on old data.
- **Retraining** from scratch on old + new data is computationally expensive and may not be feasible.

This is a "core AI" problem that has seen significant recent work. The ideal system would incorporate new knowledge without destroying existing knowledge.

Challenge 3: Distribution Shift

In supervised learning, we model the joint distribution $p(x, y)$ from a training set $\mathcal{D}_{\text{train}}$, learning $p_\theta(y|x)$. The joint can be decomposed as:

$$p(x, y) = p(y|x) \cdot p(x) = p(x|y) \cdot p(y)$$

Distribution shift occurs when the real-world distribution changes after deployment. There are three distinct types:

Types of Distribution Shift

Domain Shift — $p(x)$ changes, but $p(y|x)$ stays the same.
Example: A sentiment classifier trained on Amazon reviews deployed on Twitter. The input style changes (short text, slang), but positive words still indicate positive sentiment.

Label Shift — $p(y)$ changes, but $p(x|y)$ stays the same.
Example: A spam classifier trained on 50/50 spam/non-spam data deployed where only 5% is spam. What spam and non-spam look like hasn't changed, just how frequently each class occurs.

Concept Drift — $p(y|x)$ changes.
Example: A work-from-home predictor trained in 2019. Post-COVID, the same person with the same features has a completely different probability of working from home. The fundamental mapping from features to labels has changed because latent variables (not in x) have shifted.

Concept drift is the most dangerous type because the model's learned function is actually *wrong* — it cannot be fixed by recalibration or reweighting. New labeled data from the shifted distribution is required.

Challenge 4: Online Learning

In online learning, data arrives as a stream: $(x_1, y_1), (x_2, y_2), \ldots$ The model processes each example and updates continuously. Unlike continual learning, online learning typically does not care about forgetting old data — it focuses on performing well on the current task. It keeps updating the ML system in a loop and is specific to a single task.

Generalization and Intelligence

The lecture references François Chollet's 2020 paper "On the Measure of Intelligence," which provides a framework for thinking about what generalization really means.

Given training data from $p(x, y)$ that shifts to $p'(x, y)$ via some mechanism, the goal is to predict y from x under the new distribution. Two types of generalization are distinguished:

Two Types of Generalization

System-Aware Generalization (Developer-driven): The developer builds techniques (regularization, data augmentation, transfer learning) that help the model generalize. The human engineers the generalization.

Developer-Aware Generalization (System-driven): The system does things that were never intended or envisaged. This includes reasoning, synthesis, and creativity ("good hallucination"). The system itself discovers how to generalize.

AI and Skill Acquisition

An AI system learns a set of skills $\{S_1, S_2, S_3, \ldots\}$. The meta-learning objective can be reframed as optimizing the **speed and data efficiency of skill acquisition** — acquiring new skills and new concepts rapidly. This reframing connects meta-learning to broader questions about artificial general intelligence: can we build systems that acquire new capabilities as efficiently as humans do?

Lecture 2: From Classification to Attention-Based Few-Shot Learning (January 8)

This lecture builds the mathematical foundation for few-shot learning, starting from a simple example and progressively deriving the attention mechanism.

Motivating Example: Customer Purchase Prediction

Setup: We have a database of N customers. For each customer j:

- Feature vector: $x_j = (x_j^1, x_j^2, \ldots, x_j^K)$ where features include age, gender, address, occupation, etc. The features are heterogeneous: age is numeric (#), gender is binary (0/1), address could be a region code, occupation could be a continuous embedding $[0, 0.1]$.
- Label: $y_j \in \{0, 1\}$ where $1 =$ buy, $0 =$ no buy.

A new customer arrives with features x. We want to estimate $p(y|x)$.

Step 1: Direct Estimation with Indicator Functions

The most naive approach — find exact matches in the database:

$$\boxed{p(y|x) = \frac{\sum_i \mathbb{1}[x = x_i] \cdot y_i}{\sum_i \mathbb{1}[x = x_i]}} \tag{1}$$

where $\mathbb{1}[x = x_i]$ is the indicator function (1 if features match exactly, 0 otherwise).

Numerator: Among all exact matches, count how many bought the product (since $y_i \in \{0, 1\}$, summing y_i counts the buyers).

Denominator: Count total number of exact matches.

Problem: Curse of Dimensionality

With high-dimensional continuous features, the probability of finding an exact match is essentially zero. The denominator will be 0 for almost every new customer.

Step 2: Relaxation to Neighbors (ε-Ball / k-NN)

Instead of exact matches, find customers who are "close enough":

$$p(y|x) = \frac{\sum_{|x - x_j| \leq \varepsilon} y_j}{\sum_{|x - x_j| \leq \varepsilon} 1} \tag{2}$$

The hard indicator $\mathbb{1}[x = x_i]$ is replaced by the neighborhood condition $|x - x_j| \leq \varepsilon$. This is essentially **$k$-nearest neighbors** classification.

Remaining issue: All neighbors within the ε-ball are weighted equally. A customer who is nearly identical to the query contributes the same as one barely within the ball.

Step 3: Distance-Weighted Prediction

Replace the hard ε cutoff with a soft distance weighting. First, we need distance measures:

- **Euclidean distance:** $|x - x_i|$ — straightforward but sensitive to feature scales.
- **Cosine similarity:** Angle-based, invariant to vector magnitude.
- **Learned distance:** $d(x, x_i) = f(x)^\top f(x_i)$ — pass both through a neural network f_θ, take dot product in embedding space.

The soft weighted prediction becomes:

$$p(y|x) = \frac{\sum_i d(x, x_i) \cdot y_i}{\sum_i d(x, x_i)} \tag{3}$$

Here, y_i are one-hot vectors: $[0, 1]$ for "bought," $[1, 0]$ for "didn't buy." The output $[p_0, p_1]$ gives class probabilities.

Step 4: Softmax Formulation

Raw distance weights can be negative (dot products can be negative). We need positive weights that sum to 1. The **softmax** provides this:

$$p(y|x) = \frac{\sum_i e^{d(x,x_i)} \cdot y_i}{\sum_i e^{d(x,x_i)}} \tag{4}$$

Defining the attention weights:

$$\alpha_i = \frac{e^{d(x,x_i)}}{\sum_j e^{d(x,x_j)}} = \text{Softmax}_i(d(x, x_j)) \tag{5}$$

The prediction is:

$$\boxed{p(y|x) = \sum_i \alpha_i \cdot y_i = \boldsymbol{a}^\top Y} \tag{6}$$

where $\boldsymbol{a}$ is the vector of attention weights and Y is the matrix of training labels.

Step 5: The Attention Connection

The softmax-weighted sum is precisely the **attention mechanism**:

- The query is $f(x)$ (the test point's embedding)
- The keys are $f(x_i)$ (the training points' embeddings)
- The values are y_i (the training labels)
- Softmax over query-key dot products gives attention weights
- Weighted sum of values gives the prediction

Step 6: Generalizing with Separate Encoders

Why should the query and key use the same function? We generalize to separate encoders:

$$d(x_1, x_2) = f(x_1)^\top g(x_2) \qquad \text{(separate query and key encoders)} \tag{7}$$

$$\hat{y} = h(y) \qquad \text{(value encoder for labels)} \tag{8}$$

Step 7: The Master Equation

For a batch of M test points $\hat{X}$ and N training points X:

$$\boxed{\hat{Y} = \text{Softmax}\Big(f(\hat{X}) \cdot g(X)^\top\Big) \cdot h(Y)} \tag{9}$$

Mapping to transformer attention $\text{Attention}(Q, K, V) = \text{Softmax}(QK^\top)V$:

Component	Transformer	Few-Shot Classifier
Query (Q)	$W_Q X$	$f(\hat{X}) \in \mathbb{R}^{M \times d}$
Key (K)	$W_K X$	$g(X) \in \mathbb{R}^{N \times d}$
Value (V)	$W_V X$	$h(Y) \in \mathbb{R}^{N \times C}$
Output	$\mathbb{R}^{N \times d}$	$\hat{Y} \in \mathbb{R}^{M \times C}$

This is **cross-attention**: the test set cross-attends to the training set. Queries come from $\hat{X}$ (test), keys and values come from X, Y (training).

What is Being Learned?

We are **not** learning $p(y|x)$ directly as a parametric function. We are learning f, g, h — the *embedding functions* that define what "similarity" means. Classification happens at inference time through comparison, not through memorized weights. This is why it works for few-shot learning: when new classes arrive, we keep the same embeddings and just provide new reference examples.

Training via Representation Learning: Contrastive Loss

To train the embedding f_θ, we need similarity $f_\theta(x_i)^\top f_\theta(x_j)$ to be:

- **Large** when $y_i = y_j$ (same class → close embeddings)
- **Small** when $y_i \neq y_j$ (different class → far embeddings)

The **contrastive loss** with margin M and threshold ε:

$$\boxed{\mathcal{L}_{\text{cont}} = \sum_{\substack{i,j \\ y_i = y_j}} \text{ReLU}\big(M - f_\theta(x_i)^\top f_\theta(x_j)\big) + \sum_{\substack{i,j \\ y_i \neq y_j}} \text{ReLU}\big(f_\theta(x_i)^\top f_\theta(x_j) - \varepsilon\big)} \tag{10}$$

Term 1 (same class): If similarity $< M$, penalize (pull together). If already $\geq M$, ReLU outputs 0 (no gradient).

Term 2 (different class): If similarity $> \varepsilon$, penalize (push apart). If already $\leq \varepsilon$, ReLU outputs 0.

The gap between $\varepsilon < M$ creates a margin of safety in the embedding space. The ReLU acts as a hinge: only violations are penalized, making training focus on the hard/informative pairs.

Computational cost: $O(N^2)$ — all pairwise comparisons. This motivates later methods like triplet loss and InfoNCE that use smarter sampling strategies.

Lecture 3: Generalization Theory and Representation Learning (January 12)

Recap: Soft k-NN and Training Approaches

The lecture restates the master equation with explicit dimensions:

$$\hat{Y}_{M\times 2} = \text{Softmax}\Big(f_\theta(\hat{X}) \cdot f_\theta(X)^\top, \text{dim} = 1\Big)_{M\times N} \cdot Y_{N\times 2}$$

where dim = 1 indicates row-wise softmax (each test point's attention sums to 1).

Two approaches to training θ are contrasted:

Approach 1: Direct (End-to-End) Training

$$\min_\theta \|Y - \hat{Y}\|$$

Backpropagate through the entire softmax-attention-embedding pipeline. This is task-specific and requires both a support set and query set to compute the loss — it is inherently episodic.

Approach 2: Contrastive Training

$$\min_\theta \mathcal{L}_{\text{cont}}$$

Optimize the embedding space structure via pairwise similarity constraints. This is more general-purpose and focuses on representation quality rather than end-task performance.

Task Structure and Cross-Task Generalization

The data is organized into tasks. Each task consists of a support set and a query set, drawn from a specific classification problem (e.g., "classify shampoo buyers" or "classify shoe buyers").

The Representation Goal

The embedding $f_\theta(x) \in \mathbb{R}^d$ must produce representations that **generalize across tasks**. A single f_θ is shared across all tasks — it is not re-learned for each one. The representation must capture features that are useful for *any* task from the task distribution, not just one specific task.

For this to work, the new tasks must be **related** to the training tasks. They must share the same modality, domain, and type of discriminative features. The specific classes can be entirely new, but the underlying structure must be similar.

The No Free Lunch Theorem

No Free Lunch Theorem

Consider a task $f : X \to Y$ where $X = \{0,1\}^N$ and $Y = \{0,1\}$. There are $|\{0,1\}|^{|\{0,1\}^N|} = 2^{2^N}$ possible functions. Given a training set S, a model g can satisfy $g(x) = y$ for all $(x, y) \in S$, but there exist exponentially many functions that agree on S but disagree on any test point outside S.

Conclusion: No model can generalize across ALL possible tasks without additional assumptions.

This theorem has profound implications for meta-learning:

- Generalization requires an **inductive bias** — an assumption that the tasks we will encounter are *related* to the tasks we train on.
- In meta-learning, this assumption is that test tasks come from the same *task distribution* as training tasks.
- LLMs generalize to a "large" number of tasks (due to the breadth of their pretraining distribution) but not ALL tasks — consistent with No Free Lunch.

Architectures for the Embedding Function f_θ

Two categories depending on whether labels are used:

Unsupervised / Self-Supervised (Using Only X)

- **Compressive Autoencoder** — bottleneck forces informative compression
- **Denoising Autoencoder** — reconstruct clean input from corrupted version
- **Diffusion Models** — iterative denoising process
- **Variational Autoencoder (VAE)** — stochastic latent space with generative capability

Sequence Models (for Sequential Inputs)

- **RNN / LSTM** — sequential processing with hidden state
- **Transformers** — parallel processing with self-attention

Task Representation

Tasks themselves can be represented as vectors, encoding properties like dataset statistics, class distributions, or summary features. This enables measuring task similarity, predicting transfer performance, or conditioning models on task descriptions.

Lecture 4: Representation Learning Methods (January 15)

This lecture provides a comprehensive tour of all major methods for learning the embedding function f_θ.

Problems with Standard Contrastive Loss

Restating the contrastive loss:

$$\mathcal{L} = \mathbb{1}_{y_i=y_j} \operatorname{ReLU}\big(M - f_\theta(x_i)^\top f_\theta(x_j)\big) + \mathbb{1}_{y_i\neq y_j} \operatorname{ReLU}\big(f_\theta(x_i)^\top f_\theta(x_j) - \varepsilon\big)$$

Issue with Term 1 ($y_i = y_j$): The strongest "same class" signal comes when $x_i = x_j$ (same data point). But $f_\theta(x_i)^\top f_\theta(x_i) = \|f_\theta(x_i)\|^2$ is trivially large. The network learns nothing from self-comparisons.

Even with distinct same-class points ($x_i \neq x_j$, $y_i = y_j$), the loss only constrains pairwise dot products. It does not force f to learn semantically meaningful features — degenerate solutions (e.g., mapping everything to the same point) can satisfy the constraints without producing useful representations.

Contrastive Loss Using Noise

Noise-Based Positive Pairs

Instead of relying on class labels, create positive pairs by **data augmentation**:

1. Take a data point x_i
2. Create a noisy version $x_i + \eta$ where η is random noise
3. The pair $(x_i, x_i + \eta)$ forms a positive pair (same "identity")
4. Different data points form negative pairs

This eliminates the need for labels entirely — it is a **self-supervised** approach.

The connection to **kernel density estimation** is:

$$p(x) = \sum_{i=1}^{N} e^{-|x-x_i|^2} \tag{11}$$

Each training point contributes a Gaussian bump. The noise-based contrastive loss learns an embedding that respects this density structure — points in high-density regions map nearby, points in low-density regions map far.

This is the foundational idea behind SimCLR, MoCo, BYOL, and all modern self-supervised contrastive methods.

Reconstructive Representation Learning

A fundamentally different approach: learn f_θ by encoding then decoding, and measuring reconstruction quality:

$$x \xrightarrow{f_\theta} z \xrightarrow{g_\theta} \hat{x}, \qquad |x - \hat{x}| < \varepsilon \tag{12}$$

If z can be decoded back to x, it must contain the essential information. Four variants are presented:

Variant 1: Compressive Autoencoder (Vanilla AE)

$$x \in \mathbb{R}^m \xrightarrow{f_\theta} z \in \mathbb{R}^d \xrightarrow{g_\theta} \hat{x} \in \mathbb{R}^m, \qquad d \ll m \tag{13}$$

The **bottleneck** ($d \ll m$) forces the encoder to compress. If a 784-dimensional image must be represented in 32 dimensions, the encoder must discover the most informative features (edges, textures, shapes). The decoder proves these features are sufficient by reconstructing the original.

Loss: $\mathcal{L} = \sum_{i=1}^{N} \|x_i - \hat{x}_i\|^2$

Variant 2: Denoising Autoencoder

$$x \to x + \eta \xrightarrow{f_\theta} z \xrightarrow{g_\theta} \hat{x}, \qquad |x - \hat{x}| < \varepsilon \tag{14}$$

The input is **corrupted with noise** η, but the reconstruction target is the *clean* original x. This is harder than vanilla AE — the encoder must learn to ignore noise and extract the true signal. This produces more robust representations that capture the underlying data manifold rather than surface-level details.

Variant 3: Diffusion Models

Instead of a single noising step, diffusion applies noise **iteratively**:

$$x_0 \to \begin{pmatrix} \sqrt{\alpha}\, x_0 \\ 1-\alpha \end{pmatrix} = x_1 \to \begin{pmatrix} \sqrt{\alpha}\, x_1 \\ 1-\alpha \end{pmatrix} = x_2 \to \cdots \to \text{noise} \tag{15}$$

At each step t, the data is mixed with noise controlled by schedule parameter α. After many steps, the result is pure Gaussian noise. The model learns the reverse process:

$$p_\theta(x_{t-1} \mid x_t, t) \tag{16}$$

Given a noisy version and the timestep, predict the slightly less noisy version. This yields $\tilde{p}(x)$, a model of the data distribution. The intermediate features of the denoising network serve as powerful representations.

Variant 4: Variational Autoencoder (VAE)

$$x \xrightarrow{h_\theta} z = (\mu, \sigma) \to z + \eta \xrightarrow{g_\theta} \hat{x}, \qquad |\hat{x} - x| < \varepsilon \tag{17}$$

where $\eta \sim \mathcal{N}(0, I)$, so the actual latent code sampled is $\mu + \sigma \cdot \mathcal{N}(0, 1)$.

Key difference from vanilla AE: The latent space is *stochastic*. The encoder outputs distribution parameters (μ, σ), not a deterministic point. This enables:

- **Reconstruction:** Sample $\hat{z}$ from learned $\tilde{p}(z)$, decode to $\hat{x}$
- **Generation:** Sample $z \sim \mathcal{N}(0, I)$ from the prior, decode to a *new* data point

Variants:

- **CVAE** (Conditional VAE): condition generation on a class label
- **VQVAE** (Vector Quantized VAE): discrete latent codes (powers models like DALL-E)

VAE Loss Function

The encoder produces $z = f_\theta(x) = (\mu_z, \sigma_z)$. The VAE loss is:

$$\boxed{\mathcal{L}_{\text{VAE}} = \underbrace{\sum_{i=1}^{N} \|x_i - \hat{x}_i\|}_{\text{Reconstruction Loss}} - \underbrace{D(\mathcal{N}(\mu_z, \sigma_z) \parallel \mathcal{N}(0,1))}_{\text{KL Divergence (Variational Loss)}}} \tag{18}$$

The **KL divergence** between two distributions q and p is:

$$\text{KL}(q\|p) = \sum_i q(x_i) \log \frac{q(x_i)}{p(x_i)} \tag{19}$$

The VAE Tension

The two loss terms are in opposition:

- **Reconstruction loss** wants z to be maximally informative about x — high variance, spread-out latent codes that distinguish each input.
- **KL loss** wants z to look like $\mathcal{N}(0,1)$ — low variance, overlapping latent codes that look like random noise.

The balance between them determines how structured vs. how "smooth" the latent space is. More KL weight $\rightarrow$ smoother latent space, better generation, worse reconstruction. Less KL weight $\rightarrow$ sharper reconstruction, fragmented latent space, worse generation.

Sequence Architectures for f_θ

For sequential data $x_0, x_1, \ldots, x_{n-1}$ where each $x_i \in \mathbb{R}^m$:

Architecture	Key Properties
CNN (stride, dilated)	Local patterns, shift-invariant, efficient
LSTM (gating)	Long-range sequential dependencies, explicit memory cell
RNN	Basic sequential processing, vanishing gradient issues
Transformer (Attention)	Direct access between all positions, self-attention
Transformer (Cross-Attention)	Two sequences interacting — exactly the matching network mechanism

The lecture ends by tying this back to **Matching Networks** — all these representation learning methods and architectures are choices for implementing the embedding functions in the few-shot classification framework.

Lecture 5: Sequence Models, Attention, and the Meta-Learning Framework (January 19)

MLP Review and Universal Approximation

MLP: $y = f(x)$, $x \in \mathbb{R}^m$. Single layer: $\sigma(Wx + b)$. Batch: $\sigma(XW + B)$ where $X \in \mathbb{R}^{N \times m}$.

Universal Approximation Theorem

For any continuous function $g(x)$ on a compact subset of $\mathbb{R}^m$ and any desired precision $\varepsilon > 0$, there exists an MLP $f(x)$ with a single hidden layer such that $|f(x) - g(x)| < \varepsilon$ for all x in the domain.

Implication: MLPs are theoretically powerful enough to approximate any function. The challenge is finding the right parameters efficiently.

Limitation: MLPs process each input independently. They have no mechanism for handling sequential dependencies.

Recurrent Neural Networks (RNN)

For sequences $x_0, x_1, \ldots, x_{T-1}$:

$$h_t = f_\theta(x_t, h_{t-1}) \tag{20}$$

$$y_t = g(h_t) \tag{21}$$

The hidden state h_t serves as the "memory" of the network, encoding all relevant information from timesteps $0, 1, \ldots, t$. The same parameters θ are shared across all timesteps (weight tying).

Computational equivalence: With finite-precision arithmetic, RNNs are equivalent to **Finite State Machines (FSMs)**. They can recognize regular languages (e.g., a^*b^*c) but not context-sensitive languages (e.g., $a^n b^n c^n$) which require unbounded counting.

The Vanishing/Exploding Gradient Problem

Training RNNs requires backpropagation through time (BPTT). The gradient of the loss with respect to parameters involves:

$$\frac{\partial \mathcal{L}}{\partial \theta} = \frac{\partial \mathcal{L}}{\partial y} \cdot \frac{\partial g}{\partial h} \cdot \frac{\partial f}{\partial \theta} \tag{22}$$

Expanding $\partial f / \partial \theta$ recursively:

$$\frac{\partial f}{\partial \theta} = \frac{\partial f}{\partial x_t} \cdot (\ldots) + \frac{\partial f}{\partial h_{t-1}} \cdot \frac{\partial h_{t-1}}{\partial \theta} \tag{23}$$

The recursive term $\partial h_{t-1}/\partial \theta$ requires $\partial h_{t-2}/\partial \theta$, and so on. This creates a chain of multiplications by $\frac{\partial h_t}{\partial h_{t-1}} \approx W$ (the recurrent weight matrix). Over T timesteps:

- If the largest eigenvalue $|\lambda_{\max}(W)| > 1$: gradients **explode** ($W^T \to \infty$)
- If $|\lambda_{\max}(W)| < 1$: gradients **vanish** ($W^T \to 0$)

Consequence

Vanilla RNNs cannot learn long-range dependencies. Information from early timesteps is either washed out (vanishing) or causes instability (exploding). This motivated the development of gated architectures.

Long Short-Term Memory (LSTM)

The LSTM addresses vanishing gradients through a **cell state** C_t that acts as a gradient highway, and **four gates** that control information flow:

LSTM Equations

All gates receive the concatenation $[h_{t-1}, x_t]$ as input:

$$\text{Forget gate:} \quad f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \tag{24}$$

$$\text{Input gate:} \quad i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \tag{25}$$

$$\text{Candidate:} \quad \tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \tag{26}$$

$$\text{Output gate:} \quad o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \tag{27}$$

Cell state and hidden state updates:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \tag{28}$$

$$h_t = o_t \odot \tanh(C_t) \tag{29}$$

where $\odot$ denotes element-wise multiplication.

Why this solves vanishing gradients: The cell state C_t is updated via element-wise multiply (by f_t) and element-wise add (of $i_t \odot \tilde{C}_t$). There are no matrix multiplications on the $C_{t-1} \to C_t$ path. Gradients can flow through this "highway" without repeated matrix multiplication, enabling learning of long-range dependencies.

Gate intuitions:

- **Forget gate** (f_t): What to erase from memory. $f_t \approx 0$ means "forget everything," $f_t \approx 1$ means "remember everything."
- **Input gate** (i_t): What new information to write to memory.
- **Candidate** ($\tilde{C}_t$): The actual new content to potentially write.
- **Output gate** (o_t): What part of memory to expose as the hidden state.

The LSTM can handle context-sensitive patterns like $a^n b^n c^n$ because the cell state can act as an explicit counter with sufficient precision.

Gated Recurrent Unit (GRU)

The GRU simplifies the LSTM by using only **3 gates** instead of 4. The key simplification: the input gate is coupled as $(1 - f_t)$. Whatever fraction of old information you forget, you replace with new information:

$$C_t = f_t \odot C_{t-1} + (1 - f_t) \odot \tilde{C}_t \tag{30}$$

This constraint (keep + update = 1) reduces parameters while maintaining similar performance in many tasks.

Attention Mechanism

Self-attention over a sequence:

$$Y = \text{Softmax}\Big(f(X) \cdot f(X)^\top\Big) \cdot h(X) \tag{31}$$

where $h(X) \in \mathbb{R}^{m \times N}$. In the simplest form, $h(X) = WX$ where $W \in \mathbb{R}^{d \times m}$.

The architecture stacks an LSTM/RNN/GRU at the bottom to produce hidden states $h_0, \ldots, h_{n-1}$, then an attention layer on top to allow direct interactions between all positions.

Key advantage over RNNs: Attention provides *direct* access between any two positions. Position 0 can attend to position 99 without information flowing through 99 intermediate steps. No vanishing gradients over long distances.

Machine Translation: Encoder-Decoder with Cross-Attention

The machine translation architecture (e.g., Hindi → English) consists of:

Encoder: A bidirectional LSTM/Attention processes the source sequence $x_0, \ldots, x_{N-1}$ (Hindi tokens), producing hidden representations:

$$H(X) \in \mathbb{R}^{N \times d}$$

Decoder: Generates target tokens $y_0, y_1, \ldots, y_{M-1}$ (English tokens) autoregressively, maintaining decoder states $Z \in \mathbb{R}^{M \times d}$.

Cross-Attention: The decoder queries the encoder:

$$\text{Cross-Attn} = \text{Softmax}\Big(f(Z) \cdot g(H)^\top\Big) \tag{32}$$

For each English word being generated, the attention weights indicate which Hindi words are relevant. This learned alignment replaces the need for explicit word-by-word translation tables.

Teacher forcing: During training, the decoder receives the ground-truth previous token (not its own prediction) as input. This stabilizes training but creates a train-test discrepancy.

Transformers

The transformer uses self-attention as its core operation at each layer:

$$\text{Output} = \text{Softmax}\Big(f(X) \cdot g(X)^\top\Big) \cdot h(X) \tag{33}$$

with separate learned projections f, g, h (the Q, K, V projections), applied in parallel across all positions. Multiple layers are stacked for depth.

Causal Masking: For autoregressive language modeling (predicting next token), a triangular mask prevents each position from attending to future positions:

$$\text{Mask}_{ij} = \begin{cases} 0 & \text{if } j \leq i \quad \text{(allowed)} \\ -\infty & \text{if } j > i \quad \text{(blocked)} \end{cases}$$

This masking is *only* needed for language models. Classification (BERT-style) and encoding use full bidirectional attention.

The Formal Meta-Learning Framework

Meta-Learning Definitions

A **task** T is defined as:

$$T = \{\mathcal{D}_{\text{train}}, \mathcal{D}_{\text{test}}\} \tag{34}$$

where $\mathcal{D}_{\text{train}} = (X_{\text{train}}, Y_{\text{train}})$ is the **support set** and $\mathcal{D}_{\text{test}} = (X_{\text{test}}, Y_{\text{test}})$ is the **query set**.

K-shot, W-way scenario: K labeled samples from each of W classes. The support set has $K \times W$ examples total.

Meta-training set: $\{T_0, T_1, \ldots, T_{N-1}\}$ — many *related* tasks for training.
Meta-testing set: $\{\hat{T}_0, \hat{T}_1, \ldots, \hat{T}_{M-1}\}$ — new tasks (different classes) for evaluation.

Example: Omniglot dataset. Handwritten characters from many alphabets: Hindi, English (a, b), Japanese, etc. Each character class has a few handwritten examples. A 5-way 1-shot task: given 5 previously unseen characters with 1 example each, classify new handwriting samples.

The Meta-Learning Philosophy

Learning to Learn

Goal G: "Learn from a small number of examples."

Idea of meta-learning: If G is the goal, then **train for that goal**. Create many few-shot tasks. For each task, ask the ML system to learn — to classify from few examples. Over many such episodes, the system gets better at the *process of learning*.

This is the fundamental distinction: standard ML optimizes for performance on a *single task*. Meta-learning optimizes for the *ability to quickly learn new tasks*.

Lecture 6: Matching Networks and MAML (January 22)

This lecture presents two complete meta-learning algorithms: the metric-based **Matching Networks** and the optimization-based **MAML**.

Matching Networks: Full Formalization

The complete Matching Networks equation with train/test notation:

$$\boxed{\hat{Y}_{\text{test}} = \text{Softmax}\Big(f_\phi(X_{\text{test}}) \cdot g_\theta(X_{\text{train}})^\top\Big) \cdot Y_{\text{train}}} \tag{35}$$

Dimensions:

- $f_\phi(X_{\text{test}}) \in \mathbb{R}^{M \times d}$ — M test points embedded by query encoder
- $g_\theta(X_{\text{train}})^\top \in \mathbb{R}^{d \times N}$ — N support points embedded by key encoder
- $Y_{\text{train}} \in \mathbb{R}^{N \times C}$ — one-hot support labels over C classes
- $\hat{Y}_{\text{test}} \in \mathbb{R}^{M \times C}$ — predicted class probabilities for test points

This is **Soft Nearest Neighbors (SoftNN)** — a differentiable, attention-based nearest neighbor classifier.

Training Loss: Cross-Entropy

$$\mathcal{L}_{\text{CE}} = \text{CE}(\hat{Y}_{\text{test}}, Y_{\text{test}}) = -\text{Sum}\Big(Y_{\text{test}} \odot \log \hat{Y}_{\text{test}}\Big) \tag{36}$$

The element-wise product $Y_{\text{test}} \odot \log \hat{Y}_{\text{test}}$ selects the log-probability of the correct class for each test point (since Y_{test} is one-hot). Summing and negating gives the negative log-likelihood. Minimizing this pushes the model to assign maximum probability to the correct class.

Episodic Training Loop

Matching Networks Training Algorithm

While not converged:

1. **Sample task** $T_k = \{\mathcal{D}^k_{\text{train}}, \mathcal{D}^k_{\text{test}}\}$ from the meta-training set
2. **Forward pass:** Compute $\hat{Y}_{\text{test}} = \text{Softmax}(f_\phi(X_{\text{test}}) \cdot g_\theta(X_{\text{train}})^\top) \cdot Y_{\text{train}}$
3. **Compute loss:** $\mathcal{L}_{T_k} = \text{CE}(\hat{Y}_{\text{test}}, Y_{\text{test}})$
4. **Meta-update:** $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}_{T_k}$

Each iteration trains on a *different* task. The model never specializes to one task — it learns the *process* of comparing and classifying.

Context-Aware Matching Networks

In the basic version, $f_\theta(x_{\text{test}})$ and $g_\theta(x_{\text{train}})$ embed each point independently. The upgrade: make embeddings **conditioned on the support set**.

$$\text{Test embedding:} \quad f_\theta\big(x_{\text{test}},\ g'(\mathcal{D}_{\text{train}})\big) \tag{37}$$

$$\text{Support embedding:} \quad g_\theta\big(x_{\text{train}},\ \mathcal{D}_{\text{train}}\big) \tag{38}$$

Here g' is a **set encoder** that processes $\mathcal{D}_{\text{train}}$ into a summary vector. This context vector is fed into both encoders, allowing them to know what the current task is about.

Why Context-Awareness Helps

Consider two tasks: "dog vs. car" and "dog vs. wolf." The features needed are completely different (shape vs. fur texture). If the encoder *sees* the support set, it knows which discriminative features to extract. Without context, it must embed everything into a single general-purpose space, which is suboptimal for any specific task.

In the full Matching Networks paper (Vinyals et al., 2016):

- g_θ processes the support set with a **bidirectional LSTM** — each support point's embedding is aware of all other support points.
- f_θ processes the query with an **LSTM + attention over the support set** — the query embedding is conditioned on what's available.

General Meta-Learning Loop

The generic meta-learning algorithm that encompasses all approaches:

General Meta-Learning Loop

While training:

1. **Sample Task** T_k
2. **ADAPT** model to T_k:
   - For Matching Networks: condition encoders on $\mathcal{D}_{\text{train}}$
   - For MAML: gradient descent on $\mathcal{D}_{\text{train}}$ (next section)
3. **Predict** $\hat{Y}_{\text{test}}$ using adapted model
4. **Compute Loss** on $\mathcal{D}_{\text{test}}$
5. **Meta-Update:** $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$

Matching Networks characteristics:

- Adaptation mechanism: distance metric conditioned on $\mathcal{D}_{\text{train}}$
- No weight updates at test time — only forward passes
- Requires the specific attention-based architecture

MAML: Model-Agnostic Meta-Learning

A completely different approach to meta-learning, introduced by Finn et al. (2017).

MAML — Core Idea

"We want a model to do well on a few-shot task by one or more gradient descent steps on $\mathcal{D}_{train}$."

Instead of learning an embedding for comparison, learn an **initialization** θ of model parameters such that a few gradient descent steps on any new task's support set produces good performance on that task's query set.

Setup: A model $f_\theta(x) \to y$ (any differentiable model). θ represents the model's initial parameters — the "starting point" in parameter space.

The MAML Algorithm

MAML Algorithm

θ = *initialization of weights* (the meta-parameter)

While not converged (batch of tasks):
Sample Task $T_k = \{\mathcal{D}^k_{\text{train}}, \mathcal{D}^k_{\text{test}}\}$
Inner Loop (Adaptation):

1. Initialize: $\theta' = \theta$
2. For $s = 1$ to S:

$$\mathcal{L}^I = \text{CE}\big(Y_{\text{train}},\ f_{\theta'}(X_{\text{train}})\big) \qquad \text{(inner loss)}$$
$$\theta' \leftarrow \theta' - \varepsilon \nabla_{\theta'} \mathcal{L}^I \qquad \text{(inner update)}$$

Outer Loop (Meta-Update):

$$\mathcal{L}^O = \text{CE}\big(Y_{\text{test}},\ f_{\theta'}(X_{\text{test}})\big) \qquad \text{(outer loss)}$$
$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}^O \qquad \text{(meta-update)}$$

Understanding the Two Loops

Inner loop (task-specific adaptation): Starting from the shared initialization θ, take S gradient steps on the support set. This produces θ' — parameters specialized for this particular task. This is like "fine-tuning" for a few steps.

Outer loop (meta-learning): Evaluate how well the adapted θ' performs on the query set. Then update the initialization θ to make future adaptations more effective.

Critical Subtlety: Second-Order Gradients

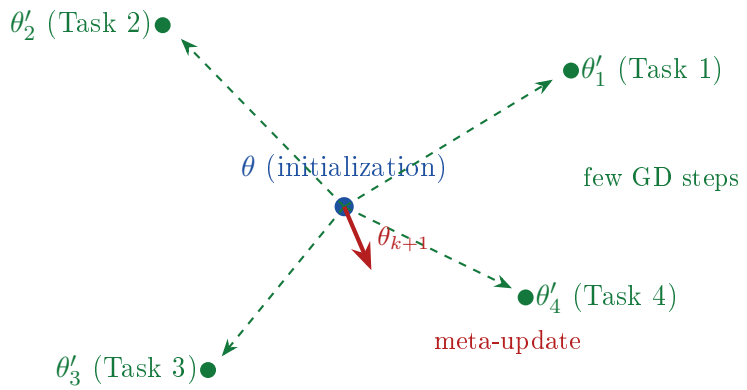
The outer update computes $\nabla_\theta \mathcal{L}^O$, not $\nabla_{\theta'} \mathcal{L}^O$. Since θ' is a function of θ (through the inner loop gradient steps), this requires **differentiating through the optimization process itself**:

$$\nabla_\theta \mathcal{L}^O(\theta') = \nabla_\theta \mathcal{L}^O\Big(\theta - \varepsilon \nabla_\theta \mathcal{L}^I(\theta)\Big)$$

This involves second-order derivatives (gradients of gradients), which is computationally expensive but is what gives MAML its power. First-order approximations (FOMAML, Reptile) drop these second-order terms for efficiency with only modest performance loss.

Geometric Interpretation

The MAML optimization landscape can be visualized as follows:



θ sits at a point in parameter space from which all task-specific optima $\theta'_1, \theta'_2, \ldots$ are reachable within a few gradient steps. It is not optimal for any single task, but it is *close enough to all of them* that brief fine-tuning works. The meta-update moves θ to an even better starting point.

Matching Networks vs. MAML: Comprehensive Comparison

Aspect	**Matching Networks**	**MAML**
Adaptation mechanism	Reads support set, conditions embeddings via attention	Gradient descent on support set (actual weight updates)
Parameters at test time	*Unchanged* — only forward passes	*Actually updated* ($\theta \to \theta'$) via inner loop
What is learned	A good *embedding space* for comparison	A good *initialization* for fast adaptation
Core mechanism	Similarity / attention	Optimization / gradient descent
Architecture requirements	Requires softmax-attention pipeline	**Any** differentiable model (model-agnostic)
Inner loop computation	Forward pass only	Forward + backward passes (S steps)
Test-time speed	Fast (single forward pass)	Slower (S gradient steps per task)
Gradient order	First-order only	Second-order (or first-order approx.)
Meta-learning family	*Metric-based*	*Optimization-based*

Summary: The Complete Arc

Lecture 1 (Jan 5): Why meta-learning? Less data, continual learning, distribution shift, generalization, skill acquisition.

↓

Lecture 2 (Jan 8): Customer prediction → k-NN → soft distance → softmax attention → the master equation $\hat{Y} = \text{Softmax}(f(\hat{X})g(X)^\top)h(Y)$.

↓

Lecture 3 (Jan 12): Training the embedding: direct vs. contrastive loss. No Free Lunch theorem. Architectures for f_θ.

↓

Lecture 4 (Jan 15): Representation learning: noise-contrastive, compressive AE, denoising AE, diffusion, VAE. Sequence architectures.

↓

Lecture 5 (Jan 19): MLP → RNN → LSTM/GRU → Attention → Cross-Attention → Transformers. Formal meta-learning setup: tasks, episodes, K-shot W-way.

↓

Lecture 6 (Jan 22): Two complete solutions — **Matching Networks** (metric-based, attention) and **MAML** (optimization-based, gradient descent).

The course builds a complete framework for few-shot learning from first principles:

1. **Motivation:** Real-world ML challenges (less data, shifting distributions, continual learning) demand systems that can learn quickly from few examples.
2. **Foundation:** Starting from naive classification (exact matching), through k-NN, through distance-weighted voting, the course derives the attention mechanism as a natural solution to few-shot classification.
3. **Theory:** The No Free Lunch theorem establishes that generalization requires structure — tasks must be related. This justifies the task-distribution assumption central to meta-learning.
4. **Representations:** Multiple methods for learning embeddings (contrastive, reconstructive, generative) provide the feature extraction capability needed for meaningful similarity computation.
5. **Architectures:** From MLPs to Transformers, the course surveys the computational tools available for implementing meta-learning systems.
6. **Algorithms:** Two fundamentally different approaches emerge:
   - **Metric-based (Matching Networks):** Learn a similarity function; classify by comparison at test time. No weight updates needed.
   - **Optimization-based (MAML):** Learn a good initialization; adapt by gradient descent at test time. Model-agnostic.

Both approaches are trained episodically — sampling tasks, adapting, evaluating, and meta-updating — embodying the core principle: to learn to learn from few examples, *practice learning from few examples.*